Authentication and Authorization

Contents

Authentication and Authorization **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 3
- References . . . 3
- Related . . . 3

Authentication and Authorization

Scope

Identity, sessions, and access control. Threat handling that is not identity-specific is in `../application-security/STANDARD.`
Design notes: `brain/architecture/auth-and-identity.md`.

Official requirements

Subject V.1, V.5 and V.6 — all graded:

- Registration by email+password **or** social account (Facebook / Google).
- Linking a social account to an existing account.
- Email verification for email+password signup, and a forgot-password flow.
- Profile data in three visibility tiers: public, friends-only, private.
- An authenticated user reaches their own data and never another user's.
- Brute-force and session-hijacking protections must exist, and remaining risks must be documented.

External baselines: **OWASP ASVS 5.0.0 Level 2** for chapters V6 Authentication, V7 Session Management, V8 Authorization, V9 Self-contained Tokens and V10 OAuth and OIDC — the committed scope is `../../../owasp/ASVS-CONFORMANCE.md`. Plus NIST SP 800-63B for password and verifier rules, and RFC 9700 OAuth 2.0 security best current practice.

Project policy

Passwords

Algorithm: **BCrypt** via `BCrypt.Net-Next` — `brain/decisions/ADR-0007-password-hashing-bcrypt.md`.

Rule	Value
Work factor (cost)	**12 minimum**, read from configuration, stored inside the hash
Salt	generated per password by the library — never reused, never hand-rolled
Accepted length	12 to 64 characters

Rule	Value
Hard input limit	**72 bytes UTF-8** — rejected at validation, never silently truncated
Rehash	on successful login, if the stored cost is below the current setting
Storage	the full BCrypt string (algorithm, cost, salt, digest) in one column

- **BCrypt truncates input at 72 bytes.** Two different long passwords would otherwise hash identically. The 64-character and 72-byte validation rules exist for exactly this reason and are not optional. NIST SP 800-63B asks verifiers to accept at least 64 characters, so this satisfies the requirement and removes the truncation case entirely.
- No composition rules, no forced rotation — per NIST SP 800-63B.
- Check submitted passwords against a breached-password list where practical.
- The work factor is tuned on the target host: high enough to cost real time, low enough not to become a denial-of-service vector. Login rate limiting stays mandatory regardless.
- Never MD5, never a bare SHA-family digest, never a hand-rolled scheme.
- Never log, never return, never include a password or a hash in any response or trace.

Tokens and sessions

- Short-lived JWT access token (15 minutes) signed with a strong asymmetric or HMAC key from `.env`.
- Refresh tokens are opaque, stored hashed server-side, rotated on every use, and revocable.
- Reuse of an already-rotated refresh token means theft: revoke the whole family and force re-login.
- Access tokens carry `sub`, `iat`, `exp`, `jti` and nothing sensitive. A JWT is signed, not secret.
- Logout revokes the refresh token server-side. Client-side deletion alone is not logout.
- Password reset and email change invalidate all existing refresh tokens.

Email verification and reset

- Tokens are single-use, high-entropy, short-TTL (verification 24 h, reset 1 h), stored hashed.
- **Expiry is checked at the point of use, every time.** A background sweep deletes expired rows, but security never depends on that sweep running (`brain/decisions/ADR-0024-expiry-and-cleanup.md`).
- Reset responses are identical whether or not the account exists. No enumeration.

Social login

- **OpenID Connect**, not bare OAuth 2.0 — `brain/decisions/ADR-0008-oidc-social-login.md`. An OAuth access token says what the bearer may do, not who they are; using one as proof of identity is the classic federated-login vulnerability.
- The device runs the authorization code flow with PKCE; the server validates the **ID token** against the provider's JWKS, checking `iss`, `aud`, `exp`, `iat` and `nonce`. A client-supplied profile is never trusted.
- One provider identity maps to at most one local user.
- Linking requires an already-authenticated session.
- See `../../50-integration/social-oauth/STANDARD.md`.

Authorisation

Two gates, both mandatory, both server-side:

1. **Ownership** — the resource belongs to the caller, or the caller is an authorised participant.
2. **Visibility and licence** — evaluated from `Visibility` and `LicenceMode` on every request and every SignalR group join.

- Implemented as an authorisation pipeline behaviour, not scattered `if` statements in handlers.

- Profile responses are filtered **per field** by the requester's relationship (self / friend / stranger). Never send a full profile and filter on the client.
- Object-level checks happen on every request, including `GET` by id. Broken object-level authorisation is the most common API vulnerability.
- A resource the caller may not see returns `404`, not `403`.

Rate limiting

Per IP **and** per account on: login, registration, password reset, verification resend, vote casting. Progressive backoff on repeated login failure. Response `429` with `Retry-After`.

Implementation: the built-in `Microsoft.AspNetCore.RateLimiting` middleware with a `PartitionedRateLimiter` — one partition per account, one per IP (`brain/decisions/ADR-0021-backend-dependency-set.md`). State is held **in process**, which is correct for a single instance and would not be shared across several.

Checklist

- ☐ BCrypt cost >= 12; password 12-64 characters and <= 72 bytes enforced in validation
- ☐ Rehash on login when the stored cost is below the current setting
- ☐ ID token validated server-side: signature, `iss`, `aud`, `exp`, `nonce`
- ☐ Refresh-token rotation with reuse detection
- ☐ Reset and verification tokens hashed, single-use, expiring
- ☐ Ownership **and** licence checked server-side on every request and hub join
- ☐ Profile filtered per field, server-side
- ☐ Rate limits on every credential endpoint
- ☐ Remaining accepted risks written down for defence

References

- OWASP ASVS 5.0.0 — vendored at `docs/owasp/upstream/asvs-5.0.0/`, scope in `docs/owasp/ASVS-CONFORMANCE.md`
- OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_S
- OpenID Connect Core 1.0 — https://openid.net/specs/openid-connect-core-1_0.html
- NIST SP 800-63B — https://pages.nist.gov/800-63-3/sp800-63b.html
- RFC 9700 OAuth 2.0 Security Best Current Practice — https://www.rfc-editor.org/rfc/rfc9700
- OWASP API Security Top 10 — https://owasp.org/API-Security/

Related

`../application-security/STANDARD.md` · `../secrets-management/STANDARD.md` · `../../50-integration/social-oau`